SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – MCQs

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO3:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 3 – MCQs
Weightage:	10% of CIA	**Total Marks:**	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	LOKESH S	**Reg. No.:**	192572151
Section / Batch:	CSE(AI) / 2ND YEAR	**Date:**	08/08/2026

Instructions to Students

- Answer all questions carefully. Each question carries 2 marks. Total: 100 marks.
- Analyze each question and select the correct answer.
- Apply concepts correctly to solve problem-based MCQs.
- Manage your time efficiently to complete all 50 questions.

Question Type	Focus Area	Questions
MCQs	Analyze and apply Divide and Conquer Algorithms for Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication.	Q1 -Q50

SECTION A – MCQs

MCQ Link :

Code of Conduct

I LOKESH S (19257215) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: LOKESH S

RUBRICS FOR CALCULATION

Online Quiz / MCQ Test					
Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Conceptual Understanding	30%	Demonstrates thorough understanding; answers all questions accurately	Shows good understanding with few errors	Partial understanding; several incorrect responses	Lacks understanding; majority of answers incorrect
Accuracy of Answers	25%	All answers are correct	Most answers are correct with minor mistakes	Some correct answers; frequent errors	Mostly incorrect answers
Application of Knowledge	20%	Correctly applies concepts to problem-based questions	Applies concepts with minor errors	Limited ability to apply concepts	Unable to apply concepts
Time Management	15%	Completes quiz well within time efficiently	Completes on time with minor delays	Requires extra time or rushes through questions	Unable to complete within given time
Question Interpretation	10%	Interprets all questions correctly and responds appropriately	Misinterprets a few questions	Often misinterprets questions	Unable to understand questions

Mark Evaluations:

No. of MCQ Questions	Correct Answers	Wrong/Not Answers	Total Marks (100)
50			

Faculty In-charge	Course Coordinator	Head of Department
Name: _____ Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

DEBUGGING & OPTIMISATION TASK

1. Debugging Selection Sort Code

Given Code

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] > arr[min_idx]:  # ERROR
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Errors Identified

The main logical error is in the comparison condition:

if arr[j] > arr[min_idx]:

Selection Sort in ascending order must find the smallest element in the unsorted portion. Therefore, the condition should be:

if arr[j] < arr[min_idx]:

Another issue is that the loop runs until n. When i = n-1, there is no remaining unsorted portion, so that iteration is unnecessary.

Root Cause

The code is actually selecting the largest element instead of the smallest element. As a result, the array is arranged incorrectly for ascending sorting.

Debugging Steps

1. Set min_idx = i.
2. Compare every remaining element with arr[min_idx].
3. Update min_idx when a smaller element is found.
4. Swap only after finding the minimum element.
5. Stop the outer loop at n-1.
6. Avoid unnecessary swapping when min_idx == i.

Optimized Corrected Code

```
def selection_sort(arr):
    n = len(arr)

    # Find the minimum element for each position
    for i in range(n - 1):
        min_idx = i

        # Search for the smallest element
        # in the unsorted portion
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j

        # Swap only if a smaller element was found
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]

    return arr

# Example
arr = [64, 25, 12, 22, 11]
print(selection_sort(arr))
```

Output

[11, 12, 22, 25, 64]

Complexity Analysis

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$
- Space Complexity: $O(1)$

Selection Sort always scans the remaining elements, so the number of comparisons remains quadratic. The optimization only reduces unnecessary swaps, not the asymptotic comparison complexity.

2. Debugging Linear Search Code

Given Code

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return -1  # ERROR
    return i  # ERROR
```

Errors Identified

There are two major errors:

1. When the key is found, the code returns -1.
2. When the key is not found, the code returns i, which is incorrect because i may contain the last loop index.

The function should return the index when the element is found and -1 after the entire loop finishes without finding it.

Root Cause

The return statements are reversed logically.

Correct behavior:

Element found → return index

Element not found → return -1

Debugging Steps

1. Traverse the array from left to right.
2. Compare every element with key.
3. Immediately return i when a match is found.
4. If the loop completes, return -1.

Corrected Code

```
def linear_search(arr, key):
    # Check each element sequentially
    for i, value in enumerate(arr):
        if value == key:
            return i

    # Key was not found
    return -1
```

```
# Example
arr = [10, 20, 30, 40, 50]


print(linear_search(arr, 30))
print(linear_search(arr, 100))
```

Output

```
2
-1
```

Complexity Analysis

- Best Case: O(1) — key is the first element.
- Average Case: O(n)
- Worst Case: O(n) — key is at the end or absent.
- Space Complexity: O(1)

Linear Search must potentially examine every element, so its worst-case time complexity is O(n).

3. Debugging Closest Pair Code

Given Code

```
import math

def closest_pair(points):
    min_dist = 0  # ERROR

    for i in range(len(points)):
        for j in range(i+1, len(points)):
            dist = math.sqrt(
                (points[i][0]-points[j][0])**2 +
                (points[i][1]-points[j][1])**2
            )

            if dist > min_dist:  # ERROR
                min_dist = dist

    return min_dist
```

Errors Identified

There are two major logical errors.

Error 1: Incorrect initialization

min_dist = 0

Distance is always non-negative. Therefore, no normal positive distance will be smaller than 0.

It should be initialized to infinity:

min_dist = float('inf')

Error 2: Incorrect comparison

The code uses:

if dist > min_dist:

For the closest pair, we need the smallest distance, so it must be:

if dist < min_dist:

Root Cause

The code is effectively searching for the largest distance rather than the smallest distance.

Optimization

The square root operation is unnecessary while comparing distances. We can compare squared distances instead.

Instead of:

math.sqrt(dx**2 + dy**2)

we compare:

dx**2 + dy**2

The square root can be calculated only at the end.

Corrected Optimized Code

```
import math

def closest_pair(points):
    # Need at least two points
    if len(points) < 2:
        return None

    min_dist_sq = float('inf')

    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            # Calculate coordinate differences
            dx = points[i][0] - points[j][0]
            dy = points[i][1] - points[j][1]

            # Compare squared distances
            dist_sq = dx * dx + dy * dy

            if dist_sq < min_dist_sq:
                min_dist_sq = dist_sq

    # Return actual Euclidean distance
    return math.sqrt(min_dist_sq)
```

Example

points = [(0, 0), (3, 4), (1, 1), (7, 8)]

print(closest_pair(points))

Output

1.4142135623730951

The closest pair is (0,0) and (1,1).

Complexity Analysis

There are approximately n(n-1)/2 pairs.

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$
- Space Complexity: O(1)

Avoiding square roots improves the constant factor but does not change the $O(n^2)$ complexity.

4. Debugging Convex Hull Logic

Given Code

```
def orientation(p, q, r):
    return (q[1]-p[1])*(r[0]-q[0]) - \
           (q[0]-p[0])*(r[1]-q[1])  # SIGN ISSUE
```

Error Identified

The orientation formula is written in a confusing form using vectors from p to q and q to r.

The standard cross-product orientation formula is:

```
(q[0] - p[0]) * (r[1] - p[1]) - \
(q[1] - p[1]) * (r[0] - p[0])
```

Orientation Meaning

For three points p, q, and r:

- Positive → Counter-clockwise
- Negative → Clockwise
- Zero → Collinear

The sign convention can be reversed if the algorithm consistently uses the opposite convention. The real problem is using the orientation result incorrectly when deciding whether a point belongs to a hull edge.

Corrected Orientation

```
def orientation(p, q, r):
    value = ((q[0] - p[0]) * (r[1] - p[1]) -
             (q[1] - p[1]) * (r[0] - p[0]))

    if value > 0:
        return 1       # Counter-clockwise
    elif value < 0:
        return -1      # Clockwise
    else:
        return 0       # Collinear
```

Brute-Force Convex Hull Logic

An edge (p, q) belongs to the convex hull if all other points lie on the same side of the line pq.

Optimized Corrected Code

```
def orientation(p, q, r):
    """
    Determine the orientation of three points.
    Returns:
        1  -> counter-clockwise
        -1 -> clockwise
        0  -> collinear
    """
    value = ((q[0] - p[0]) * (r[1] - p[1]) -
             (q[1] - p[1]) * (r[0] - p[0]))

    if value > 0:
        return 1
    elif value < 0:
        return -1
    return 0



def convex_hull(points):
    n = len(points)

    if n <= 1:
        return points

    hull = []

    for i in range(n):
        for j in range(i + 1, n):

            side = 0
            valid_edge = True
```

```
            for k in range(n):
                if k == i or k == j:
                    continue

                current = orientation(points[i], points[j], points[k])

                if current == 0:
                    continue

                if side == 0:
                    side = current
                elif current != side:
                    valid_edge = False
                    break

            if valid_edge:
                if points[i] not in hull:
                    hull.append(points[i])
                if points[j] not in hull:
                    hull.append(points[j])

    return hull


# Example
points = [
    (0, 0),
    (4, 0),
    (4, 4),
    (0, 4),
    (2, 2)
]
```

print(convex_hull(points))

Complexity Analysis

The brute-force method checks:

- Every pair of points → $O(n^2)$
- Every other point for each pair → $O(n)$

Therefore:

Time Complexity = $O(n^3)$

Space Complexity:

$O(n)$ for storing hull points.

More advanced algorithms such as Graham Scan and Monotonic Chain can solve the convex hull problem in $O(n \log n)$.

5. Debugging Binary Search Code

Given Code

```
def binary_search(arr, key):
    low, high = 0, len(arr)-1

    while low < high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            high = mid - 1  # ERROR
        else:
            low = mid + 1  # ERROR

    return -1
```

Errors Identified

Error 1: Incorrect boundary update

If:

arr[mid] < key

then the key must be on the right side.

Therefore:

low = mid + 1

is required.

Error 2: Incorrect update for larger value

If:

arr[mid] > key

the key must be on the left side.

Therefore:

high = mid - 1

is required.

The given code has these two updates reversed.

Error 3: Incorrect loop condition

The condition:

while low < high:

can skip the case where low == high.

It should be:

while low <= high:

Overflow-Safe Mid Calculation

In languages with fixed-size integers, the safer calculation is:

mid = low + (high - low) // 2

In Python, integer overflow is not normally an issue because Python integers have arbitrary precision, but this form is still standard and portable.

Corrected Code

```
def binary_search(arr, key):
    """
    Performs binary search on a sorted array.
    Returns the index of key if found, otherwise -1.
    """

    low = 0
    high = len(arr) - 1

    while low <= high:
        # Overflow-safe calculation
        mid = low + (high - low) // 2

        if arr[mid] == key:
            return mid

        elif arr[mid] < key:
            # Search in the right half
            low = mid + 1

        else:
```

```
            # Search in the left half
            high = mid - 1

    return -1


# Example
arr = [10, 20, 30, 40, 50, 60]

print(binary_search(arr, 40))
print(binary_search(arr, 25))
```

Output

```
3
-1
```

Complexity Analysis

Binary Search eliminates approximately half of the search space after every comparison.

- Best Case: O(1)
- Average Case: O(log n)
- Worst Case: O(log n)
- Space Complexity: O(1) for iterative implementation.

Important: The input array must already be sorted.

6. Debugging Bubble Sort Code

Given Code

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(n-1):
            if arr[j] < arr[j+1]:  # ERROR
                arr[j], arr[j+1] = arr[j+1], arr[j]

    return arr
```

Errors Identified

Error 1: Incorrect comparison

For ascending order, adjacent elements should be swapped when:

arr[j] > arr[j + 1]

The given condition:

arr[j] < arr[j + 1]

sorts the array in descending order.

Error 2: Unnecessary comparisons

The largest element reaches its correct position after every pass.

Therefore, the inner loop can be reduced to:

range(n - i - 1)

Error 3: No early termination

If the array becomes sorted before all passes are completed, the algorithm should stop.

A swapped flag can be used.

Corrected Optimized Code

```
def bubble_sort(arr):
    """
    Sorts an array in ascending order using optimized Bubble Sort.
    """

    n = len(arr)
```

```
    for i in range(n - 1):
        swapped = False

        # Last i elements are already sorted
        for j in range(n - i - 1):

            if arr[j] > arr[j + 1]:
                # Swap adjacent elements
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # If no swap occurred, array is already sorted
        if not swapped:
            break

    return arr


# Example
arr = [64, 34, 25, 12, 22, 11, 90]

print(bubble_sort(arr))
```

Output

[11, 12, 22, 25, 34, 64, 90]

Complexity Analysis

- Best Case: O(n) due to the swapped flag.
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$
- Space Complexity: O(1)

The optimization makes Bubble Sort significantly faster for already or nearly sorted arrays.

7. Debugging Insertion Sort Code

Given Code

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j > 0 and arr[j] > key:  # ERROR
            arr[j+1] = arr[j]
            j -= 1

        arr[j] = key  # ERROR

    return arr
```

Errors Identified

Error 1: Incorrect loop condition

The code uses:

while j > 0

This prevents comparison with the element at index 0.

It should be:

while j >= 0

Error 2: Incorrect insertion position

After shifting elements, the key must be placed at:

arr[j + 1] = key

The given code uses:

arr[j] = key

which overwrites the wrong element.

Root Cause

Insertion Sort shifts larger elements to the right until it finds the correct position for key.

After the loop ends, j has moved one position before the correct insertion location. Therefore, the key belongs at j + 1.

Corrected Code

```
def insertion_sort(arr):
```

```
    """
    Sorts an array in ascending order using Insertion Sort.
    """

    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        # Shift larger elements one position to the right
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        # Insert key into its correct position
        arr[j + 1] = key

    return arr


# Example
arr = [12, 11, 13, 5, 6]

print(insertion_sort(arr))
```

Output

```
[5, 6, 11, 12, 13]
```

Complexity Analysis

- Best Case: O(n) — already sorted.
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$ — reverse sorted.
- Space Complexity: O(1)

Insertion Sort is particularly efficient for small or nearly sorted arrays.

8. Debugging String Matching Code

Given Code

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)

    for i in range(n - m):
        for j in range(m):
            if text[i+j] != pattern[j]:
                break

        if j == m-1:
            print("Match at index", i)
```

Error Identified

The major error is the outer loop:

range(n - m)

Python's range() excludes the upper limit.

The last valid starting position is:

n - m

Therefore, the loop must be:

range(n - m + 1)

Example

If:

text length = 5

pattern length = 2

valid starting positions are:

0, 1, 2, 3

But:

range(5 - 2)

produces:

0, 1, 2

So the final possible match is skipped.

Additional Edge Case

If the pattern is empty, it is better to define a clear behavior rather than relying on the loop logic.

Corrected Code

```
def string_match(text, pattern):
    """
    Finds all occurrences of pattern in text
    using brute-force string matching.
    """

    n = len(text)
    m = len(pattern)

    if m == 0:
        return []

    matches = []

    # +1 is required to include the last valid position
    for i in range(n - m + 1):

        match = True

        for j in range(m):
            if text[i + j] != pattern[j]:
                match = False
                break

        if match:
            matches.append(i)

    return matches
```

Example

text = "AABAACAADAABAABA"

pattern = "AABA"

print(string_match(text, pattern))

Output

[0, 9, 12]

Complexity Analysis

Let:

- n = length of text
- m = length of pattern

In the worst case, the algorithm checks m characters at approximately n positions.

- Best Case: O(n)
- Average Case: O(nm)
- Worst Case: O(nm)
- Space Complexity: O(k) for storing k matches.

The brute-force algorithm can be optimized further using algorithms such as KMP, which provides O(n + m) worst-case matching.

9. Debugging Maximum Element Search Code

Given Code

```
def find_max(arr):
    max_val = 0  # ERROR

    for i in range(len(arr)):
        if arr[i] < max_val:  # ERROR
            max_val = arr[i]

    return max_val
```

Errors Identified

Error 1: Incorrect initialization

max_val = 0

This fails when all elements are negative.

For example:

[-5, -10, -2]

The correct maximum is -2, but initializing to 0 incorrectly produces 0.

The safest approach is to initialize using the first element.

Error 2: Incorrect comparison

The code uses:

arr[i] < max_val

For maximum search, we need:

arr[i] > max_val

Root Cause

The code is effectively trying to find a smaller value while starting from an inappropriate fixed value.

Edge Case

An empty array has no maximum value. Therefore, the function should handle it explicitly.

Corrected Code

```
def find_max(arr):
    """
    Returns the maximum element in an array.
```

```
    Raises ValueError if the array is empty.
    """

    if not arr:
        raise ValueError("Array cannot be empty")

    max_val = arr[0]

    # Compare each remaining element
    for value in arr[1:]:
        if value > max_val:
            max_val = value

    return max_val


# Example
arr = [-10, -25, -3, -40, -7]

print(find_max(arr))
```

Output

-3

Complexity Analysis

Every element must potentially be examined.

- Best Case: O(n)
- Average Case: O(n)
- Worst Case: O(n)
- Space Complexity: O(1)

The algorithm requires a single traversal of the array.

10. Debugging Pair Sum Code (Brute Force)

Given Code

```
def pair_sum(arr, target):
    for i in range(len(arr)):
        for j in range(len(arr)):
            if i != j and arr[i] + arr[j] == target:
                return (i, j)


    return None
```

Analysis

The given code can actually detect valid pairs, so there is no fundamental correctness error in the pair condition.

However, it performs unnecessary work because it checks both:

(i, j)

and:

(j, i)

For example:

(0, 1)

(1, 0)

represent the same pair.

It also checks the same index combinations unnecessarily before rejecting them with:

i != j

Optimization

Instead of:

for j in range(len(arr)):

we can start j from i + 1.

This guarantees:

j > i

and therefore automatically prevents:

- Using the same element twice.
- Checking duplicate pair orders.

Corrected Optimized Code

```
def pair_sum(arr, target):
```

```
    """
    Finds two different elements whose sum equals target.

    Returns:
        A tuple containing the two indices.
        None if no valid pair exists.
    """

    n = len(arr)

    for i in range(n):
        # Only check elements after i
        for j in range(i + 1, n):

            if arr[i] + arr[j] == target:
                return (i, j)

    return None


# Example
arr = [2, 7, 11, 15]
target = 9

print(pair_sum(arr, target))
```

Output

(0, 1)

Because:

arr[0] + arr[1]

= 2 + 7

= 9

Further Optimization

The brute-force approach can be improved to O(n) using a hash table.

```
def pair_sum_optimized(arr, target):
    """
    Finds a pair of indices whose values add up to target.
    Uses a hash table for O(n) average-case time.
    """

    seen = {}

    for i, value in enumerate(arr):
        complement = target - value

        if complement in seen:
            return (seen[complement], i)

        seen[value] = i

    return None



# Example
arr = [2, 7, 11, 15]
target = 9

print(pair_sum_optimized(arr, target))
```

Output

(0, 1)

Complexity Analysis

Brute-Force Version

- Best Case: O(1) if the first pair matches.
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

- Space Complexity: O(1)

Hash Table Version

- Best Case: O(n)
- Average Case: O(n)
- Worst Case: O(n) average practical performance
- Space Complexity: O(n)

The hash-table solution trades additional memory for substantially better search performance.